TRABALHO PRÁTICO FINAL

Treinamento de Modelo de Visão Computacional – Detecção de Capacetes

Aluno: Samuel de Araujo Freire

Arquivos do projeto: https://github.com/ProjectFreire/trabalhovisao

1. Descrição do Problema

O objetivo deste trabalho é desenvolver um modelo de visão computacional capaz de detectar a presença ou ausência de capacetes em imagens. Esse tipo de aplicação é extremamente relevante em contextos de segurança do trabalho, especialmente em ambientes industriais e de construção civil, onde o uso de equipamentos de proteção individual (EPIs) é obrigatório.

A utilização de técnicas de visão computacional permite automatizar a fiscalização do uso de capacetes, reduzindo a dependência de supervisão humana e aumentando a eficiência na identificação de situações de risco.

Para resolver esse problema, foi utilizado um modelo baseado em Deep Learning, especificamente o YOLO (You Only Look Once), que é amplamente utilizado em tarefas de detecção de objetos.

2. Dataset Utilizado

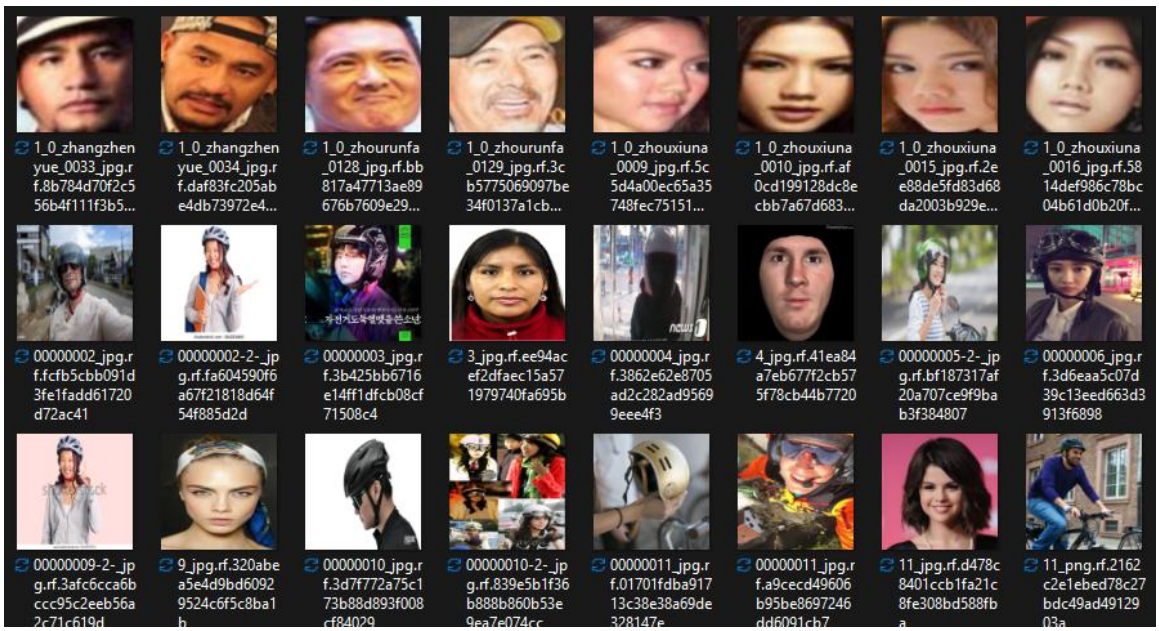
Foi utilizado um dataset público contendo imagens anotadas com duas classes: "helmet" e "non-helmet". O dataset foi obtido por meio da plataforma Roboflow Universe e já se encontrava estruturado no formato YOLO.

Classes do dataset:

- helmet: indica a presença de capacete
- non-helmet: indica a ausência de capacete

Distribuição aproximada do dataset:

- Treino: ~1950 imagens
- Validação: ~550 imagens
- Teste: ~550 imagens



3. Análise Crítica do Dataset

O dataset apresenta boa diversidade de cenários, com diferentes pessoas, ângulos e condições de iluminação. No entanto, algumas limitações podem ser observadas:

- Possível predominância de ambientes industriais específicos;
- Diferença na quantidade de exemplos entre as classes;
- Baixa diversidade de tipos de capacete.

Apesar disso, o dataset se mostrou adequado para o treinamento de um modelo robusto, permitindo bons resultados.

4. Treinamento do Modelo

Foi utilizado o modelo YOLOv8n, uma versão leve e eficiente da arquitetura YOLO. O treinamento foi realizado localmente no VS Code, utilizando CPU.

Configurações utilizadas:
- Épocas: 30
- Tamanho da imagem: 640
- Batch size: 8

O modelo foi treinado utilizando um dataset previamente dividido em conjuntos de treino, validação e teste.

```
Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
22/30         0G     0.7378     0.4555      1.276         11        640: 100% ━━━━━━━━━━━━ 244/244 5.0s/it 20:22
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 35/35 3.4s/it 2:01
             all        558        766      0.914      0.885      0.939      0.697

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
23/30         0G     0.7185     0.4422      1.267          9        640: 100% ━━━━━━━━━━━━ 244/244 5.1s/it 20:55
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 35/35 3.4s/it 1:60
             all        558        766      0.936      0.881      0.946      0.696

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
24/30         0G     0.7017      0.425      1.248         12        640: 100% ━━━━━━━━━━━━ 244/244 4.9s/it 19:51
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 35/35 3.4s/it 1:60
             all        558        766      0.936      0.874      0.942      0.701

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
25/30         0G      0.713     0.4248      1.246          9        640: 100% ━━━━━━━━━━━━ 244/244 4.9s/it 19:52
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 35/35 2.2s/it 1:16
             all        558        766      0.945      0.891      0.952       0.71

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
26/30         0G      0.693     0.3954       1.24         15        640: 100% ━━━━━━━━━━━━ 244/244 4.1s/it 16:41
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 35/35 3.3s/it 1:56
             all        558        766      0.941        0.9      0.955      0.718

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
27/30         0G     0.6818     0.3871      1.239          8        640: 100% ━━━━━━━━━━━━ 244/244 4.8s/it 19:21
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 35/35 3.2s/it 1:52
             all        558        766      0.928      0.901       0.95      0.713

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
28/30         0G     0.6708     0.3886      1.221          8        640: 100% ━━━━━━━━━━━━ 244/244 4.7s/it 19:03
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 35/35 3.2s/it 1:53
             all        558        766      0.942      0.884      0.949      0.719

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
29/30         0G     0.6621     0.3801      1.215         10        640: 100% ━━━━━━━━━━━━ 244/244 4.7s/it 19:08
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 35/35 3.3s/it 1:55
             all        558        766      0.932      0.903      0.954      0.716

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
30/30         0G     0.6612     0.3677       1.21         11        640: 100% ━━━━━━━━━━━━ 244/244 4.7s/it 19:02
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 35/35 3.2s/it 1:53
             all        558        766      0.949      0.891      0.954       0.72
```

5. Resultados Experimentais

Os resultados obtidos demonstraram um excelente desempenho do modelo.

Métricas principais:
- Precisão (Precision): 0.949
- Recall: 0.891
- mAP@0.5: 0.954
- mAP@0.5:0.95: 0.72

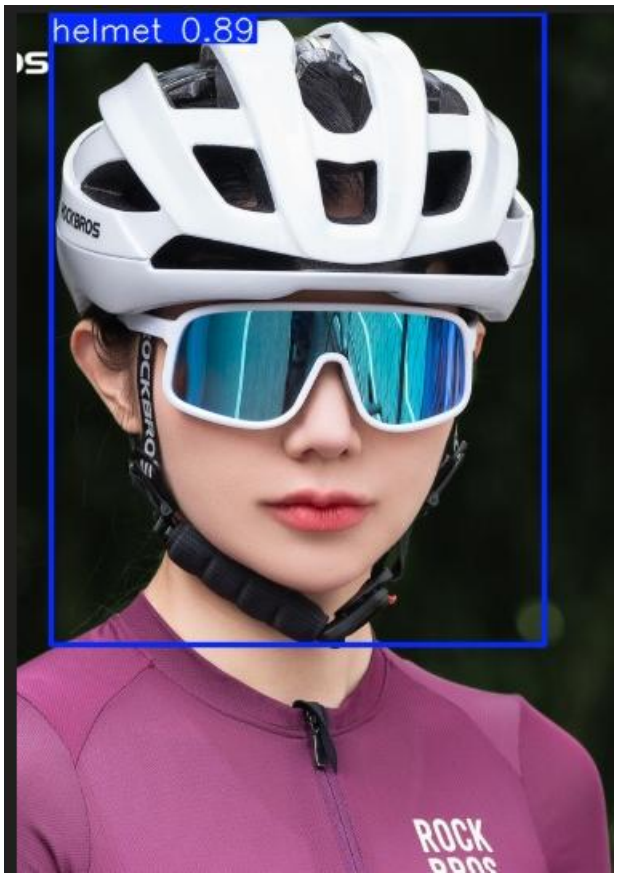
Esses resultados indicam que o modelo é altamente eficiente na detecção de capacetes.

```
30 epochs completed in 9.531 hours.
Optimizer stripped from C:\Users\samue\runs\detect\runs\treino_capacete\weights\last.pt, 6.2MB
Optimizer stripped from C:\Users\samue\runs\detect\runs\treino_capacete\weights\best.pt, 6.2MB

Validating C:\Users\samue\runs\detect\runs\treino_capacete\weights\best.pt...
Ultralytics 8.4.41  Python-3.12.10 torch-2.11.0+cpu CPU (Intel Core i5-8300H 2.30GHz)
Model summary (fused): 73 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 35/35 2.6s/it 1:32
                   all        558        766      0.949      0.891      0.954       0.72
                helmet        215        390       0.91      0.862      0.932      0.655
            non-helmet        356        376      0.988       0.92      0.977      0.785
Speed: 3.8ms preprocess, 151.0ms inference, 0.0ms loss, 0.7ms postprocess per image
```

6. Inferência do Modelo

Foi realizado um teste de inferência utilizando uma imagem externa ao dataset. O modelo foi capaz de detectar corretamente a presença de um capacete em uma imagem de contexto esportivo, demonstrando boa capacidade de generalização.



7. Conclusão

O modelo desenvolvido apresentou excelente desempenho na tarefa proposta, atingindo altos valores de precisão e mAP. Além disso, demonstrou capacidade de generalização ao identificar capacetes em contextos diferentes do dataset de treinamento.

Como trabalhos futuros, recomenda-se a utilização de datasets mais diversos e o treinamento em GPU para reduzir o tempo de processamento.